

算法部分

kd树

找二维平面的离原点最近的k近邻

```

import heapq

class KdNode:
    def __init__(self, point, split, left=None, right=None):
        self.point = point
        self.split = split
        self.left = left
        self.right = right

class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        # 1. 构建 KD 树
        def build_tree(data, depth):
            if not data:
                return None

            axis = depth % 2 # 2维空间, 只有 0 和 1
            data.sort(key=lambda x: x[axis])
            mid = len(data) // 2
            return KdNode(
                point=data[mid],
                split=axis,
                left=build_tree(data[:mid], depth + 1),
                right=build_tree(data[mid+1:], depth + 1)
            )

        root = build_tree(points, 0)

        # 2. 搜索 K 个最近邻
        # heap 中存储的是 (-距离, 点), 因为 Python 的 heapq 是小顶堆,
        # 用负数可以模拟大顶堆, 方便弹出距离最远的点
        self.max_heap = []

        def get_dist(p):
            return p[0]**2 + p[1]**2 # 省略开方以优化性能

        def search(node):
            if not node:
                return

            dist = get_dist(node.point)
            axis = node.split

            # 维护大小为 k 的大顶堆

```

```

if len(self.max_heap) < k:
    heapq.heappush(self.max_heap, (-dist, node.point))
elif dist < -self.max_heap[0][0]:
    heapq.heapreplace(self.max_heap, (-dist, node.point))

# 决定搜索方向
target_val = 0 # 因为是离原点(0,0)的距离, 目标点值永远是0
if target_val < node.point[axis]:
    near, far = node.left, node.right
else:
    near, far = node.right, node.left

search(near)

# 剪枝: 如果另一侧还可能存在更近的点, 则进入另一侧
# 这里的“墙”是 node.point[axis], 目标点在 0
# 距离墙的平方距离是 (node.point[axis] - 0)^2
if len(self.max_heap) < k or (node.point[axis]**2 < -self.max_heap[0][0]):
    search(far)

search(root)
return [p for d, p in self.max_heap]

```

快速排序(双指针法)

```
def quick_sort(arr, low, high):
    """
    快速排序主函数（递归入口）
    arr: 待排序的列表, low: 当前处理子数组的起始索引, high: 当前处理子数组的结束索引
    """
    # 递归终止条件: 当子数组长度为 0 或 1 时 (low >= high), 无需排序
    if low < high:
        # 1. 分区操作: 将数组分为两部分, 左边 <= pivot, 右边 >= pivot
        # 返回 pivot 最终所在的索引位置
        pivot_idx = partition(arr, low, high)

        # 2. 递归排序 pivot 左边的子数组
        quick_sort(arr, low, pivot_idx - 1)

        # 3. 递归排序 pivot 右边的子数组
        quick_sort(arr, pivot_idx + 1, high)

def partition(arr, low, high):
    """
    分区函数（双指针法 / 双向扫描）。逻辑：
    1. 选取第一个元素作为基准值（pivot）。
    2. left 指针从左向右找大于 pivot 的数。
    3. right 指针从右向左找小于 pivot 的数。
    4. 交换这两个数，直到指针相遇。
    5. 最后将 pivot 放到相遇位置（right），完成分区。

    返回: right: pivot 最终所在的索引
    """
    # 选取子数组的第一个元素作为基准值
    pivot = arr[low]

    # 初始化双指针
    # left 从基准值的下一个位置开始，向右扫描
    left = low + 1
    # right 从子数组末尾开始，向左扫描
    right = high

    while True:
        # 只要 left 没越界 且 当前元素 <= pivot, 就继续向右移
        # 目的: 找到第一个 > pivot 的元素
        while left <= right and arr[left] <= pivot:
            left += 1
        # 只要 right 没越界 且 当前元素 >= pivot, 就继续向左移
        # 目的: 找到第一个 < pivot 的元素
        while right >= left and arr[right] >= pivot:
            right -= 1
        # 如果 left 和 right 没有相遇, 就交换元素
        if left < right:
            arr[left], arr[right] = arr[right], arr[left]
    # 最后将 pivot 放到相遇位置 (right)
    arr[low], arr[right] = arr[right], arr[low]
    return right
```

```

    left += 1

# 只要 right 没越界 且 当前元素 >= pivot, 就继续向左移
# 目的: 找到第一个 < pivot 的元素
while left <= right and arr[right] >= pivot:
    right -= 1

# --- 判断是否交换 ---
if left <= right:
    # 如果 left 和 right 没有交错, 说明找到了逆序对
    # 交换 arr[left] (大于pivot) 和 arr[right] (小于pivot)
    arr[left], arr[right] = arr[right], arr[left]
else:
    # 如果 left > right, 说明指针已交错, 扫描结束
    break

# --- 基准值归位 ---
# 循环结束后, right 指向的是最后一个 <= pivot 的位置
# 将基准值 (arr[low]) 与 arr[right] 交换
# 此时, right 左侧都 <= pivot, 右侧都 >= pivot
arr[low], arr[right] = arr[right], arr[low]

# 返回基准值的最终索引, 供递归使用
return right

```

KMP (Knuth-Morris-Pratt)

重点关注如何更新LPS (Longest Proper Prefix which is also Suffix)

- 当你有一个很长的文本串 S (主串), 和一个较短的模式串 P , 你想知道 P 是否在 S 中出现过, 以及出现的位置在哪里。

对字符串 pattern 的某个位置 i :

$lps[i]$ = 从开头到 i 位置的子串中, 最长的那个既是前缀、又是后缀, 且不包含整个字符串的长度。

```
"""
```

compute_lps 函数用于计算模式字符串的LPS表。LPS表是一个数组，其中的每个元素表示模式字符串中当前位置之前的子串的最长前缀后缀的长度。该函数使用了两个指针 length 和 i，从模式字符串的第二个字符开始遍历。

```
"""
```

```
def compute_lps(pattern):
```

```
    """
```

计算pattern字符串的最长前缀后缀 (Longest Proper Prefix which is also Suffix) 表

:param pattern: 模式字符串

:return: lps表

```
    """
```

```
    m = len(pattern)
```

```
    lps = [0] * m # 初始化lps数组
```

```
    length = 0 # 当前最长前后缀长度
```

```
    for i in range(1, m): # 注意i从1开始, lps[0]永远是0
```

```
        while length > 0 and pattern[i] != pattern[length]:
```

```
            length = lps[length - 1] # 回退到上一个有效前后缀长度
```

```
        if pattern[i] == pattern[length]:
```

```
            length += 1
```

```
        lps[i] = length
```

```
    return lps
```

```
def kmp_search(text, pattern):
```

```
    n = len(text)
```

```
    m = len(pattern)
```

```
    if m == 0:
```

```
        return 0
```

```
    lps = compute_lps(pattern)
```

```
    matches = []
```

```
    # 在 text 中查找 pattern
```

```
    j = 0 # 模式串指针
```

```
    for i in range(n): # 主串指针
```

```
        while j > 0 and text[i] != pattern[j]:
```

```
            j = lps[j - 1] # 模式串回退
```

```
        if text[i] == pattern[j]:
```

```
            j += 1
```

```
        if j == m:
```

```
            matches.append(i - j + 1) # 匹配成功
```

```
            j = lps[j - 1] # 查找下一个匹配
```

```
    return matches
```

- 找最小循环元：假设一个字符串的长度为 L (1-based 计数)，其对应的 LPS 数组最后一位为 $\text{lps}[L-1]$ 。若满足 $L \bmod (L - \text{lps}[L-1]) == 0$ ，则：
该字符串由一个长度为 $d = L - \text{lps}[L-1]$ 的子串重复构成。
该子串即为最小循环元，重复次数 $K = L / d$ 。
 $\text{next}[i]$ = 前 i 个字符中，最长相等前缀与后缀的长度

```
def is_repeated_pattern(s: str) -> bool:
    n = len(s)
    next = [0] * (n + 1)
    j = 0
    for i in range(2, n + 1):
        while j > 0 and s[j] != s[i-1]:
            j = next[j]
        if s[j] == s[i - 1]:
            j += 1
        next[i] = j

    p = n - next[n]
    return n % p == 0 and n != p
```

关键路径

边活动 (Activity On Edge, AOE) 网

AOE网络中的最长路径被称为关键路径，把关键路径上的活动称为关键活动。

有向无环图 (DAG)，节点代表事件或里程碑，边代表活动，并且每条边有一个权重，表示完成该活动所需的时间。

step1: 计算最早开始时间 (Earliest Start Time, EST)

使用拓扑排序遍历图

$\text{EST}[v] = \max(\text{EST}[v], \text{EST}[u] + \text{weight}(u, v))$

step2: 计算最晚开始时间 (Latest Start Time, LST)

反向遍历拓扑排序后的图

$\text{LST}[v] = \min(\text{LST}[v], \text{LST}[u] - \text{weight}(v, u))$

```

from collections import defaultdict, deque
class CriticalPath:
    def __init__(self, n):
        self.n = n # 节点数
        self.graph = defaultdict(list)
        self.in_degree = [0] * n
    def add_edge(self, u, v, w):
        self.graph[u].append((v, w))
        self.in_degree[v] += 1
    def critical_path(self):
        # 拓扑排序
        queue = deque()
        for i in range(self.n):
            if self.in_degree[i] == 0:
                queue.append(i)
        topo_order = []
        ve = [0] * self.n # 最早发生时间
        while queue:
            u = queue.popleft()
            topo_order.append(u)
            for v, w in self.graph[u]:
                if ve[u] + w > ve[v]:
                    ve[v] = ve[u] + w
                self.in_degree[v] -= 1
                if self.in_degree[v] == 0:
                    queue.append(v)
        # 逆向计算最晚发生时间
        vl = [ve[topo_order[-1]]] * self.n
        for u in reversed(topo_order):
            for v, w in self.graph[u]:
                if vl[v] - w < vl[u]:
                    vl[u] = vl[v] - w
        # 找关键活动
        critical_edges = []
        for u in range(self.n):
            for v, w in self.graph[u]:
                e = ve[u]
                l = vl[v] - w
                if e == l:
                    critical_edges.append((u, v, w))
        return ve, vl, critical_edges

```


最小生成树（MST:Minimum Spanning Tree）算法

对于一个连通的加权无向图，最小生成树是该图的一个子图，它包含图中所有的顶点，并且是一棵边权值之和最小的树（即任意两个顶点之间有且只有一条路径，且边数为 $V - 1$ ）。

- Prim算法：用于找到连接所有顶点的最小生成树。
step1:从任意点出发：把一个起始点加入“已选集合”。
step2:寻找最短连接：在所有连接“已选集合”与“未选集合”的边中，找到权值最小的那条。
step3:吸纳新成员：将这条边连接的那个未选顶点加入“已选集合”。
step4:循环往复：重复上述过程，直到所有顶点都被包含。

```
import heapq
def prim(n, adj):
    """
    n: 顶点数量
    adj: 邻接表, 格式为 {u: [(weight, v), ...]}
    """
    # mst_weight 存储最小生成树的总权重
    mst_weight = 0
    # visited 记录节点是否已经加入生成树
    visited = [False] * n
    # pq 是优先队列, 存储格式为 (weight, to_node)
    pq = [(0, 0)] # 从顶点 0 开始, 权重为 0
    nodes_count = 0
    while pq and nodes_count < n:
        weight, u = heapq.heappop(pq)
        # 如果点已经访问过, 跳过
        if visited[u]:
            continue
        # 将点标记为已访问, 并累加权重
        visited[u] = True
        mst_weight += weight
        nodes_count += 1
        # 遍历当前点的所有邻居
        for next_weight, v in adj[u]:
            if not visited[v]:
                heapq.heappush(pq, (next_weight, v))
    # 如果加入的点数等于 n, 说明生成了完整的树
    return mst_weight if nodes_count == n else -1
```

- Kruskal算法 / 并查集：用于找到连接所有顶点的最小生成树，适用于边集合已经给定的情况。
总是选择权值最小的边，只要这条边不会和已经选中的边形成环，通常使用并查集快速判断“是否

形成环”。

```
class UnionFind:
    def __init__(self, n):
        self.parent = [i for i in range(n)]
    def find(self, i):
        if self.parent[i] == i:
            return i
        self.parent[i] = self.find(self.parent[i])
        return self.parent[i]
    def union(self, i, j):
        root_i = self.find(i)
        root_j = self.find(j)
        if root_i != root_j:
            self.parent[root_i] = root_j
            return True
        return False

def kruskal(n, edges):
    # 按权重从小到大排序
    edges.sort()
    uf = UnionFind(n)
    mst_weight = 0
    mst_edges = []
    for weight, u, v in edges:
        # 尝试合并：如果合并成功，说明没有环
        if uf.union(u, v):
            mst_weight += weight
            mst_edges.append((u, v, weight))
        # 如果已经找够了 n-1 条边，可以提前结束
        if len(mst_edges) == n-1:
            break
    return mst_weight, mst_edges
```

判断图是否有环

- 无向图

法一：DFS + visited + parent

使用 DFS（深度优先搜索）遍历图。

每次 DFS 时，记录当前节点的“父亲节点”。

如果访问到了已经访问过的节点，且不是当前节点的父亲节点，说明存在环。

```

def has_cycle_undirected(graph):
    visited = set()
    def dfs(node, parent):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                if dfs(neighbor, node):
                    return True
            elif neighbor != parent:
                return True
        return False
    for node in graph:
        if node not in visited:
            if dfs(node, -1):
                return True
    return False
# 无向图表示（邻接表）
graph = {
    0: [1, 2],
    1: [0, 2],
    2: [0, 1]
}
print("Has cycle (undirected):", has_cycle_undirected(graph)) # 输出: True

```

法二：并查集（Union-Find）

初始每个点属于不同的集合。

每条边连接两个点，如果两个点已经在同一个集合中，说明成环。

适合稠密图，边比较多时效率较高。

- 有向图

法一：DFS + recursion stack

要用一个额外的 递归栈 记录当前路径上的节点

如果在当前 DFS 过程中再次访问到了路径上的某个节点，就说明存在环。

```

def has_cycle_directed(graph):
    visited = set()
    rec_stack = set()
    def dfs(node):
        visited.add(node)
        rec_stack.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                if dfs(neighbor):
                    return True
            elif neighbor in rec_stack:
                return True
        rec_stack.remove(node)
        return False
    for node in graph:
        if node not in visited:
            if dfs(node):
                return True
    return False

# 有向图邻接表
graph = {
    0: [1],
    1: [2],
    2: [3],
    3: [0] # 回到了0, 构成环
}
print("Has cycle (directed):", has_cycle_directed(graph)) # 输出: True

```

法二：拓扑排序只能用于有向图。

将所有入度为 0 的点加入队列，每次移除一个点并减少邻接点的入度。

最后如果还有剩余点，说明存在环（因为这些点永远无法入队）。

拓扑排序（Topological Sorting）算法

对有向无环图（DAG）进行排序的一种算法。它将图中的顶点按照一种线性顺序进行排列，使得对于任意的有向边 (u, v) ，顶点 u 在排序中出现在顶点 v 的前面。

- DFS：用于对有向无环图（DAG）进行拓扑排序。

```

def topo_sort_dag(graph):
    visited = set()
    result = []
    def dfs(u):
        if u in visited:
            return
        visited.add(u)
        # 递归访问所有邻居
        for v in graph.get(u, []):
            dfs(v)
        # 【关键点】在回溯阶段加入结果
        result.append(u)
    # 遍历图中所有节点（应对不连通的情况）
    for node in graph:
        dfs(node)
    # 返回反转后的序列
    return result[::-1]

# 示例：
# A -> B, A -> C, B -> D, C -> D
dag = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['D'],
    'D': []
}

print(topo_sort_dag(dag)) # 输出：['A', 'C', 'B', 'D']（或另一种合法的顺序）

```

- Karn算法 / BFS：用于对有向无环图进行拓扑排序。
Kahn算法的基本思想是通过不断地移除图中的入度为0的顶点，并将其添加到拓扑排序的结果中，直到图中所有的顶点都被移除。

```

from collections import deque, defaultdict
def topological_sort(graph):
    indegree = defaultdict(int)
    result = []
    queue = deque()
    # 计算每个顶点的入度
    for u in graph:
        for v in graph[u]:
            indegree[v] += 1
    # 将入度为 0 的顶点加入队列
    for u in graph:
        if indegree[u] == 0:
            queue.append(u)
    # 执行拓扑排序
    while queue:
        u = queue.popleft()
        result.append(u)
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                queue.append(v)
# 示例调用代码
graph = {
    'A': ['B', 'C'],
    'B': ['C', 'D'],
    'C': ['E'],
    'D': ['F'],
    'E': ['F'],
    'F': []
}
sorted_vertices = topological_sort(graph)
print("Topological sort order:", sorted_vertices)
# Output:
# Topological sort order: ['A', 'B', 'C', 'D', 'E', 'F']

```

强连通分量算法

强连通分量（SCC：Strongly Connected Components）是指有向图中的一个极大子图，其中任意两个节点都是相互可达的。

SCC好处在于可以缩点然后变成有向无环图（DAG）进行拓扑排序

- Kosaraju算法 / 2 DFS：用于找到有向图中的所有强连通分量。

第一次DFS：在第一次DFS中，我们对图进行标准的深度优先搜索，但是在此过程中，我们记录下顶点完成搜索的顺序。这一步的目的是为了找出每个顶点的完成时间（即结束时间）。(这个结束时间就是它遍历完所有顶点之后)

反向图：接下来，我们对原图取反，即将所有的边方向反转，得到反向图。

第二次DFS：在第二次DFS中，我们按照第一步中记录的顶点完成时间的逆序，对反向图进行DFS。这样，我们将找出反向图中的强连通分量。

这个是对的：想象两个强连通分量 C_1 和 C_2 ，原图中有一条从 C_1 到 C_2 的单向边。在第一步中， C_1 的节点会比 C_2 的节点后出栈（因为 C_1 可以走到 C_2 ）。在反向图中，边变成了从 C_2 到 C_1 。当我们从栈顶（ C_1 的节点）开始在反向图中DFS时，由于边反转了，我们无法从 C_1 走到 C_2 。这样，DFS 就会被“锁”在 C_1 内部，从而精准地提取出这个分量。

```

def kosaraju(n, adj):
    # 1. 正向 DFS, 记录完成顺序
    visited = [False] * n
    stack = []
    def dfs1(u):
        visited[u] = True
        for v in adj[u]:
            if not visited[v]:
                dfs1(v)
        stack.append(u) # 回溯时压入栈
    for i in range(n):
        if not visited[i]:
            dfs1(i)
    # 2. 创建反向图
    rev_adj = [[] for _ in range(n)]
    for u in range(n):
        for v in adj[u]:
            rev_adj[v].append(u)
    # 3. 反向 DFS, 提取 SCC
    visited = [False] * n
    sccs = []
    def dfs2(u, current_scc):
        visited[u] = True
        current_scc.append(u)
        for v in rev_adj[u]:
            if not visited[v]:
                dfs2(v, current_scc)
    while stack:
        u = stack.pop()
        if not visited[u]:
            current_scc = []
            dfs2(u, current_scc)
            sccs.append(current_scc)
    return sccs

```

- Tarjan算法（只需要一次 DFS，不需要反转图）：用于找到有向图中的所有强连通分量，在性能和常数开销上通常优于 Kosaraju

通过在深度优先搜索的过程中维护一个栈来记录已经访问过的顶点，并为每个顶点分配一个"搜索次序"（DFS编号）和一个"最低链接值"。搜索次序表示顶点被首次访问的次序，最低链接值表示从当前顶点出发经过一系列边能到达的搜索次序最早的顶点的搜索次序。

step1:从图中选择一个未访问的顶点开始深度优先搜索。

step2:为当前顶点分配一个搜索次序和最低链接值，并将其入栈。

step3:对当前顶点的每个邻接顶点进行递归深度优先搜索，如果邻接顶点尚未被访问过，则递归调用。

step4:在递归回溯的过程中，更新当前顶点的最低链接值，使其指向当前顶点和其邻接顶点之间较小的搜索次序。

step5:如果当前顶点的最低链接值等于其自身的搜索次序，那么将从当前顶点开始的栈中的所有顶点弹出，并将它们构成一个强连通分量。

```

def tarjan_scc(n, adj):
    dfn = [-1] * n      # 搜索次序
    low = [-1] * n      # 最低链接值
    stack = []          # 辅助栈
    in_stack = [False] * n
    timer = 0
    sccs = []           # 存储最终结果
    def dfs(u):
        nonlocal timer
        dfn[u] = low[u] = timer
        timer += 1
        stack.append(u)
        in_stack[u] = True
        for v in adj[u]:
            if dfn[v] == -1: # 情况 A: 邻居未访问
                dfs(v)
                low[u] = min(low[u], low[v])
            elif in_stack[v]: # 情况 B: 邻居在栈中 (回边)
                low[u] = min(low[u], dfn[v])
        # 判定强连通分量的根
        if low[u] == dfn[u]:
            current_scc = []
            while True:
                node = stack.pop()
                in_stack[node] = False
                current_scc.append(node)
                if node == u:
                    break
            sccs.append(current_scc)
    for i in range(n):
        if dfn[i] == -1:
            dfs(i)
    return sccs

# 使用示例
# n, m = 3, 3
# adj = [[1], [2], [0]] # 一个简单的环 0->1->2->0
# print(tarjan_scc(n, adj)) # 输出: [[2, 1, 0]]

```

最短路径算法

Dijkstra算法：带非负权边的图中，用于找到两个顶点之间的最短路径。（这里距离路径都是算上权重的）

在所有当前已知距离中，每次选择距离起点最近的那个节点，将它的最短距离“确定”，然后用它去尝试更新（松弛）它的邻居节点距离；由于边权非负，一旦一个节点被选为当前最小，它的距离就不可能再被改小，因此通过不断重复这个“选最近、再更新”的过程，就能得到从起点到所有节点的最短路径。

```
import heapq
def dijkstra(n, edges, start):
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        graph[u].append((v, w))
        graph[v].append((u, w))
    dist = [float('inf')] * n
    dist[start] = 0
    pq = [(0, start)] # (当前距离, 节点)
    while pq:
        current_dist, node = heapq.heappop(pq)
        # 如果当前距离不是最优的, 跳过
        if current_dist > dist[node]:
            continue
        for neighbor, weight in graph[node]:
            new_dist = current_dist + weight
            if new_dist < dist[neighbor]:
                dist[neighbor] = new_dist
                heapq.heappush(pq, (new_dist, neighbor))
    return dist
```

Bellman-Ford算法：用于处理带有负权边的图的最短路径问题。

每次迭代尝试通过已知的最短路径更新其他路径（松弛）

最多只需进行 $V-1$ 次迭代，因为最短路径最多经过 $V-1$ 个顶点

第 V 次检测是否还能更新，用于发现负权环

```

def bellman_ford(graph, V, E, src):
    # 初始化从源点到所有其他顶点的距离为无穷大
    dist = [float("inf")] * V
    dist[src] = 0
    # 对所有边进行 V - 1 次松弛操作（路径经过的边数最多为 V - 1 条）
    for _ in range(V - 1):
        for u, v, w in graph:
            if dist[u] != float("inf") and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    # 检查负权环（如果再进行一次松弛还能更新，说明存在负权环）
    for u, v, w in graph:
        if dist[u] != float("inf") and dist[u] + w < dist[v]:
            print("图中存在负权回路！")
            return None
    return dist

```

Floyd-Warshall算法：用于找到图中所有顶点之间的最短路径。

初始化一个二维数组dist，用于存储任意两个顶点之间的最短距离。初始时，dist[i][j]表示顶点i到顶点j的直接边的权重，如果i和j不直接相连，则权重为无穷大。

对于每个顶点k，在更新dist数组时，考虑顶点k作为中间节点的情况。遍历所有的顶点对(i, j)，如果通过顶点k可以使得从顶点i到顶点j的路径变短，则更新dist[i][j]为更小的值。

$dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$

重复进行上述步骤，对于每个顶点作为中间节点，进行迭代更新dist数组。最终，dist数组中存储的就是所有顶点之间的最短路径。

```

def floyd_warshall(graph):
    # graph 是一个邻接矩阵，如果两点不连通，值为 float('inf')
    # 节点数量
    n = len(graph)
    # 初始化距离矩阵 dist
    dist = [list(row) for row in graph]
    # 核心算法：三层循环
    # 注意：k（中间点）必须在最外层！
    for k in range(n):
        for i in range(n):
            for j in range(n):
                # 如果通过中间点 k 的路径更短，则更新
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist

```

图 (graph)

邻接矩阵

邻接表

关联矩阵 (点与边的矩阵)

```
class Graph:
    def __init__(self, vertices, edges):
        self.vertices = vertices
        self.edges = edges
        self.adj_matrix = self.create_adj_matrix()
    def create_adj_matrix(self):
        # Create an empty adjacency matrix
        adj_matrix = [[0] * len(self.edges) for _ in range(len(self.vertices))]
        # Fill adjacency matrix based on edges
        for i, vertex in enumerate(self.vertices):
            for j, edge in enumerate(self.edges):
                if vertex in edge:
                    adj_matrix[i][j] = 1
        return adj_matrix
    def display_adj_matrix(self):
        for row in self.adj_matrix:
            print(row)

# Example usage
if __name__ == "__main__":
    vertices = ['A', 'B', 'C', 'D']
    edges = [('A', 'B'), ('B', 'C'), ('C', 'D'), ('D', 'A')]
    graph = Graph(vertices, edges)
    print("Adjacency Matrix:")
    graph.display_adj_matrix()

#Adjacency Matrix:
#[1, 0, 0, 1]
#[1, 1, 0, 0]
#[0, 1, 1, 0]
#[0, 0, 1, 1]
```

词梯问题建图

当处理列表中的每一个单词时，将它与桶上的标签进行比较。比如，POP_使用下划线作为通配符，我们将POPE和POPS放入这一个桶中。

```
for word in all_words:
    for i, _ in enumerate(word):
        bucket = f"{word[:i]}_{word[i + 1:]}"
        buckets.setdefault(bucket, set()).add(word)
```

AVL树

AVL树一共四种情况

```

import sys
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.height = 1
        self.left = None
        self.right = None
def solve():
    data = sys.stdin.read().split()
    n = int(data[0])
    tree = None
    def get_height(node):
        if not node:
            return 0
        return node.height
    def rightrotate(node):
        root = node.left
        node1 = node.left.right
        root.right = node
        node.left = node1
        node.height = max(get_height(node.left), get_height(node.right))+1
        root.height = max(get_height(root.left), get_height(root.right))+1
        return root
    def leftrotate(node):
        root = node.right
        node1 = node.right.left
        root.left = node
        node.right = node1
        node.height = max(get_height(node.left), get_height(node.right))+1
        root.height = max(get_height(root.left), get_height(root.right))+1
        return root
    def insert(node, val):
        if not node:
            return TreeNode(val)
        if val < node.val:
            node.left = insert(node.left, val)
        else:
            node.right = insert(node.right, val)
        node.height = max(get_height(node.left), get_height(node.right))+1
        balance = get_height(node.left)-get_height(node.right)
        if balance > 1:
            if val < node.left.val:
                node = rightrotate(node)

```

```

        else:
            node.left = leftrotate(node.left)
            node = rightrotate(node)
    elif balance < -1:
        if val > node.right.val:
            node = leftrotate(node)
        else:
            node.right = rightrotate(node.right)
            node = leftrotate(node)
    return node
for i in range(1,n+1):
    tree = insert(tree,int(data[i]))
result = []
def preorder(node):
    if node:
        result.append(node.val)
        preorder(node.left)
        preorder(node.right)
preorder(tree)
print(*result)
if __name__ == "__main__":
    solve()

```


前缀树 (Trie)

```
class Trie:
    def __init__(self):
        self.tree = {}
    def insert(self, word: str) -> None:
        cur = self.tree
        for i in range(len(word)-1):
            a = word[i]
            if a in cur:
                cur = cur[a]
            else:
                cur[a] = {}
                cur = cur[a]
        if word[-1] in cur:
            cur = cur[word[-1]]
            cur[None] = {}
        else:
            cur[word[-1]] = {None: {}}
    def search(self, word: str) -> bool:
        cur = self.tree
        for t in list(word):
            if t not in cur:
                return False
            cur = cur[t]
        if None in cur:
            return True
        return False
    def startsWith(self, prefix: str) -> bool:
        cur = self.tree
        for t in list(prefix):
            if t not in cur:
                return False
            cur = cur[t]
        return True

trie = new Trie()
trie.insert("apple") # {'a': {'p': {'p': {'l': {'e': {None: {}}}}}}}
trie.search("apple"); # True
trie.search("app"); # False
trie.startsWith("app"); # True
trie.insert("app"); # {'a': {'p': {'p': {'l': {'e': {None: {}}}, None: {}}}}}
```

```
trie.search("app");      # True
trie.insert("b") # {'a': {'p': {'p': {'l': {'e': {None: {}}}}, None: {}}}}, 'b': {None:
```

线段树 (segment tree)

处理任何区间类问题

将一个大区间 $O(n)$ 的查询，拆解成若干个已经预处理好的小区间 $O(\log n)$ 的拼接。

对[1,2,3, 4]建树

[1,4],

[1,2],[3,4]

[1,1],[2,2],[3,3],[4,4]

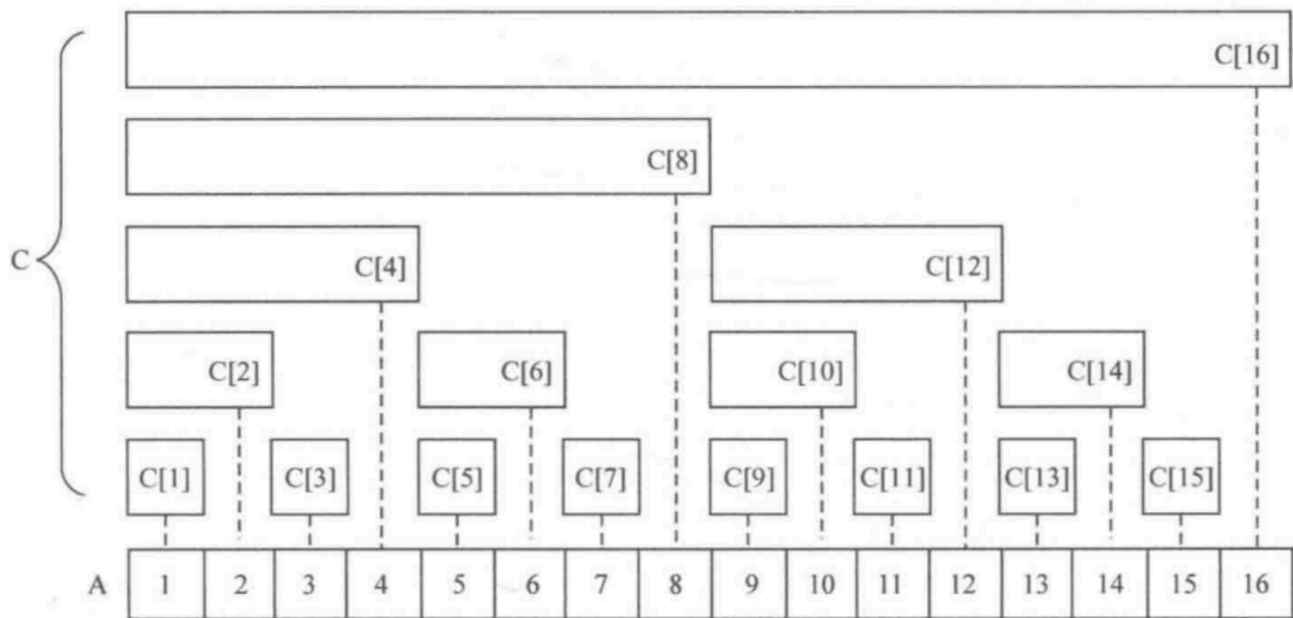
#先将序列长度扩为2的幂次再进行操作，也就是让n为2的幂

```
tree = [0] * (2*n)
def build(arr, n):
    for i in range(n):
        tree[n+i] = arr[i]
    for i in range(n-1, 0, -1):
        # 2*i 是左孩子, 2*i + 1 是右孩子
        tree[i] = tree[2*i] + tree[2*i+1]
def updateTreeNode(p, value, n):
    p = p+n
    tree[p] = value
    i = p
    while i > 1:
        i = i//2
        # 父节点等于左右两个子节点之和
        tree[i] = tree[2*i] + tree[2*i+1]
def query(l, r, n): # [l, r)
    res = 0
    l += n
    r += n
    while l < r:
        if (l % 2 != 0):
            res += tree[l]
            l += 1
        if (r % 2 != 0):
            r -= 1
            res += tree[r] #这里的r是已经被减过了的
        l = l // 2
        r = r // 2
    return res
```

树状数组 (BIT/Fenwick Tree)

单点修改和前缀和查询

动态前缀和、逆序对计数、区间更新+单点查询 (配合差分)



Binary Indexed Tree

```
class BIT:
```

```
    def __init__(self, n):
```

```
        self.size = n
```

```
        self.tree = [0] * (n + 1)
```

```
    def lowbit(self, x):
```

```
        return x & -x
```

```
    def update(self, i, delta):
```

```
        """将第 i 个元素增加 delta"""
```

```
        while i <= self.size:
```

```
            self.tree[i] += delta
```

```
            i += self.lowbit(i)
```

```
    def query(self, i):
```

```
        """查询前 i 个元素的和"""
```

```
        s = 0
```

```
        while i > 0:
```

```
            s += self.tree[i]
```

```
            i -= self.lowbit(i)
```

```
        return s
```

树 (tree)

所有给你一个表达式让你思考怎么地怎么地的题，先思考答案 **从什么树而来（大多数情况）** 或是什么 stack 而来，然后在草稿纸上画出树，再思考建树方法

关于建树，一般就是前序，中序，后序，**后缀波兰表达式** 转化，中间一般需要 stack 辅助（比如后缀波兰表达式建树就利用 stack，思路跟求值差不多）

优先队列二叉堆：先建完全二叉树，再更新的时候不停地交换，程序实现时不必用树，用列表即可
脑子卡bug一定记住要用递归

bst插入就是从根开始走

avl插入（ll型根右旋，lr型先左子树左旋再根右旋，rr型根左旋，rl型先右子树右旋再根左旋）

遍历（Traversal）：

深度优先遍历（DFS）：

前序遍历（Preorder）：根 → 左 → 右

中序遍历（Inorder）：左 → 根 → 右（BST 中序遍历结果为有序数组）

后序遍历（Postorder）：左 → 右 → 根

广度优先遍历（BFS）（层序遍历）：按层从左到右依次遍历。

```
from collections import deque
def cengxubianli(tree):
    result = []
    queue = deque([tree])
    while queue:
        for _ in range(len(queue)):
            node = queue.popleft()
            result.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    print(*result)
```

找根节点的技巧

根节点 = 出现在 all_nodes 但没出现在 child_nodes 的那个

```
root = (all_nodes - child_nodes).pop()
```

霍夫曼算法：将具有最小频率的两棵二叉树合并为一棵二叉树
测试树

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

import math
mylist = [1,2,2,None,3,None,3]
cengshu = int(math.log2(len(mylist)+1))-1
def constructtree(ceng,k):#层序，第ceng层的第k个，根在第0层
    if ceng > cengshu:
        return
    m = (1 << ceng)+k-2
    if mylist[m] != 0 and not mylist[m]:
        return
    node = TreeNode(mylist[m])
    node.left = constructtree(ceng+1,2*k-1)
    node.right = constructtree(ceng+1,2*k)
    return node
result = []
def preorder_traversal(node):
    if node:
        result.append(node.val)
        preorder_traversal(node.left)
        preorder_traversal(node.right)
tree = constructtree(0,1)
preorder_traversal(tree)
print(*result)

```

广度优先搜索(bfs)

对于图的bfs搜索，可以不用visited，用颜色来表示访问情况

一定要注意matrix的小情况和特殊情况，以及起点和终点的情况

queue内可以是多个指标，比如位置+step+魔力值这种

start不一定只有一个，可以多个同时出发

```

from collections import deque
def bfs(start):
    q = deque()
    q.append((start, 0))
    visited = {start}      # 必须用set, 否则会mle
    prev = {start: None}
    while q: # 强迫自己必须只能套一步while
        cur, step = q.popleft()
        if is_target(cur):
            path = []
            node = cur
            while node is not None:
                path.append(node)
                node = prev[node]
            path.reverse()
            print("最短路径:", path)
            return step
        for nxt in get_neighbors(cur):
            if nxt not in visited:
                visited.add(nxt)
                prev[nxt] = cur
                q.append((nxt, step+1))
    return -1 # 找不到

```

双向bfs (可以从 end 反向扩展, 反向扩展的规则和正向一样简单)

```

from collections import deque
def bidirectional_bfs(start, target, get_neighbors):
    # 若起点和终点相同，直接返回
    if start == target:
        return 0
    # 前向 & 后向队列
    q1 = deque([start])
    q2 = deque([target])
    # 前后访问集合
    visited1 = {start}
    visited2 = {target}
    steps = 0
    while q1 and q2:
        # — 优化点：永远扩展较小的那个队列 — #
        if len(q1) > len(q2):
            q1, q2 = q2, q1
            visited1, visited2 = visited2, visited1
        steps += 1
        size = len(q1)
        for _ in range(size):
            node = q1.popleft()
            for nei in get_neighbors(node):
                # 1. 如果在对方 visited 中，说明相遇了
                if nei in visited2:
                    return steps
                # 2. 如果没访问过，加入队列
                if nei not in visited1:
                    visited1.add(nei)
                    q1.append(nei)
    return -1 # 没有路径

```

并查集 (Disjoint Set)

记得编号是否需要减一与index统一

Find (path compression) 每次操作后都把沿途的每个节点直接挂到根上


```
Parent = [i for i in range(n)]
def find(i):
    if (Parent[i] == i):
        return i
    else:
        result = find(Parent[i])
        Parent[i] = result
        return result
```

```
rank = [1]*n
def union(i,j):
    i1 = find(i)
    j1 = find(j)
    if i1 == j1:
        return
    if rank[i1] < rank[j1]:
        Parent[i1] = j1
    elif rank[i1] > rank[j1]:
        Parent[j1] = i1
    else:
        Parent[j1] = i1
        rank[i1] += 1
```

```
Size = [1]*n
def union(i, j):
    i1 = find(i)
    j1 = find(j)
    if i1 == j1:
        return
    if Size[i1] < Size[j1]:
        Parent[i1] = j1
        Size[j1] += Size[i1]
    else:
        Parent[j1] = i1
        Size[i1] += Size[j1]
```

经典题目：食物链

总Time Limit: 1000ms Memory Limit: 65536kB

Description

动物王国中有三类动物A,B,C,这三类动物的食物链构成了有趣的环形。A吃B, B吃C, C吃A。
现有N个动物,以1-N编号。每个动物都是A,B,C中的一种,但是我们并不知道它到底是哪一种。
有人用两种说法对这N个动物所构成的食物链关系进行描述:

第一种说法是"1 X Y",表示X和Y是同类。

第二种说法是"2 X Y",表示X吃Y。

此人对N个动物,用上述两种说法,一句接一句地说出K句话,这K句话有的是真的,有的是假的。当一句话满足下列三条

- 1) 当前的话与前面的某些真的话冲突,就是假话;
- 2) 当前的话中X或Y比N大,就是假话;
- 3) 当前的话表示X吃X,就是假话。

你的任务是根据给定的N ($1 \leq N \leq 50,000$) 和K句话 ($0 \leq K \leq 100,000$), 输出假话的总数。

Input

第一行是两个整数N和K,以一个空格分隔。

以下K行每行是三个正整数 D, X, Y, 两数之间用一个空格隔开,其中D表示说法的种类。

若D=1,则表示X和Y是同类。

若D=2,则表示X吃Y。

Output

只有一个整数,表示假话的数目。

Sample Input

```
100 7
1 101 1
2 1 2
2 2 3
2 3 3
1 1 3
2 3 1
1 5 5
```

Sample Output

```
3
```

`type[x] % 3 ==`

0 : x 和 x的根结点 同类

1 : x 吃 x的根结点

2 : x的根结点 吃 x

$i \rightarrow i1$ (i的根): `type[i] = (i与i1是否同类的数)`

```

N,K = map(int,input().split())
Parent = [i for i in range(N)]
type = [0]*N
count = 0
def find(i):
    if (Parent[i] == i):
        return i
    else:
        result = find(Parent[i])
        type[i] = (type[i]+type[Parent[i]])%3 #这里是找以前的父节点，后面率先更新的type应该是以
        Parent[i] = result
        return result
for _ in range(K):
    D,i,j = map(int,input().split())
    if i >= N+1 or i <= 0 or j >= N+1 or j <= 0:
        count += 1
        continue
    i -= 1
    j -= 1
    if D == 1:
        i1 = find(i)
        j1 = find(j)
        if i1 == j1:
            if (type[i] - type[j]) % 3 != 0:
                count += 1
                continue
        else:
            Parent[j1] = i1
            type[j1] = (type[i]-type[j])%3
    if D == 2:
        i1 = find(i)
        j1 = find(j)
        if i1 == j1:
            if (type[i]-type[j]-1)%3 != 0:
                count += 1
                continue
        else:
            Parent[i1] = j1
            type[i1] = (type[j]-type[i]+1)%3
print(count)

```

栈 (Stack)

中缀转后缀算法

数字 扔进输出。

左括号 入栈。

操作符 把栈里比自己同等级或强的都弹出去，然后自己入栈。

右括号 栈内“大清扫”直到遇见左括号。

最后 把栈里剩下的所有东西全部倒出来。

链表 (Linked List)

链表反转

```
def reverse_linked_list(head: ListNode) -> ListNode:
    prev = None
    curr = head
    while curr is not None:
        prev = ListNode(curr.val, prev)
        curr = curr.next
    return prev
```

```
def reverse_linked_list(head: ListNode) -> ListNode:
    prev = None
    curr = head
    while curr is not None:
        next_node = curr.next # 暂存当前节点的下一个节点
        curr.next = prev      # 将当前节点的下一个节点指向前一个节点
        prev = curr           # 前一个节点变为当前节点
        curr = next_node      # 当前节点变更为原先的下一个节点
    return prev
```

合并两个排序链表

```
def merge_sorted_lists(l1, l2):
    dummy = Node(0) #dummy (哑节点 / 哨兵节点) 能避免处理链表头节点为空的边界情况
    tail = dummy
    while l1 and l2:
        if l1.data < l2.data:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    if l1:
        tail.next = l1
    else:
        tail.next = l2
    return dummy.next
```

查找链表的中间节点（快慢指针）

```
def find_middle_node(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

1->1;2->2;3->2;4->3;5->3;n->(n//2)+1

双向链表

```
class Node:
    def __init__(self, data):
        self.data = data # 节点数据
        self.next = None # 指向下一个节点
        self.prev = None # 指向前一个节点
```

循环链表可以用快慢指针去找圈

测试链表

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
def construct(mylist):
    head = ListNode()
    cur = head
    for num in mylist:
        cur.next = ListNode(num)
        cur = cur.next
    return head.next
def show(head):
    result = []
    while head:
        result.append(head.val)
        head = head.next
    return result

```

位运算

按位与 AND (&) (两个位都为 1 → 结果为 1, 否则为 0)

按位或 OR (|) (只要有一个位为 1 → 结果为 1, 两个都是 0 才为 0)

按位异或 XOR (^) (两位相同 → 0, 不同 → 1)

按位取反 NOT (~)

$\sim n$ 在二进制表示是把 n 的 0 改成 1, 1 改成 0

$-n = \sim n + 1$

n : 是一个整数, 比如 $n = 12$, 二进制是 1100

$-n$: 在计算机中用补码表示, 就是 $\sim n + 1$

$n = 00001100$

$\sim n = 11110011$

$-n = 11110100$ (相当于取反+1)

- 检查 i 是否为 2 的幂

```
if i & (i - 1) == 0:
```

- 提取一个整数在二进制表示下, 最低位的那个 1 所代表的数值。

$lowbit(x) = x \& (-x)$

语法部分

random模块

函数	功能	返回类型	取值范围 / 特点	常见用途
random()	生成随机小数	float	[0.0, 1.0)	概率判断
randint(a, b)	随机整数	int	$a \leq x \leq b$	随机页数
randrange(a, b)	随机整数	int	$a \leq x < b$	时间/事件模拟
choice(seq)	随机选一个元素	任意	序列中任选	随机位置
choices(seq, k=n)	可重复抽样	list	允许重复	概率抽样
sample(seq, k)	不重复抽样	list	无重复	抽签/组合
shuffle(seq)	原地打乱	None	修改原列表	洗牌
uniform(a, b)	随机浮点数	float	[a, b)	连续分布

运算符计算

```
opers = {'+':operator.add, '-':operator.sub, '*':operator.mul, '/':operator.truediv}
opers[运算符](a,b)
```

OOP

```
def __add__(self,Fraction1):
    qitafenzi = Fraction1.fenzi
    qitafenmu = Fraction1.fenmu
    newfenmu = qitafenmu*self.fenmu
    newfenzi = (qitafenzi*self.fenmu)+(self.fenzi*qitafenmu)
    return Fraction(newfenzi,newfenmu).huajian()
f1 = Fraction(a,b)
f2 = Fraction(c,d)
f = f1+f2
```

对类用heap的话，要利用__lt__定义大小比较关系

```
def __lt__(self, other):
    if self.weight == other.weight:
        return self.char < other.char
    return self.weight < other.weight
```

从myset中拿第一个元素

```
myset.pop()
```

nonlocal

nonlocal 关键字的作用是：在嵌套函数（函数内部的函数）中，声明一个变量不是局部变量，而是属于“外层但非全局”作用域的变量。

二进制转十进制

```
int("10",2) #2
```

深拷贝浅拷贝

浅拷贝（Shallow Copy）：`.copy()`

浅拷贝会创建一个新对象，但如果对象内部包含子对象（如列表中的列表），新对象只会引用原有的子对象。

特点：只拷贝最外层。

副作用：修改新对象中的子对象，原对象也会跟着变。

```
import copy
original = [[1, 2], 3]
shallow = original.copy() # 或者 list(original)
shallow[0][0] = 'X' # 修改嵌套的子列表
shallow[1] = 'Y' # 修改最外层元素
print(f"原对象: {original}") # 输出: [['X', 2], 3] -> 子对象变了!
print(f"浅拷贝: {shallow}") # 输出: [['X', 2], 'Y']
```

深拷贝（Deep Copy）：`copy.deepcopy()`

深拷贝会递归地拷贝对象及其内部的所有子对象。新对象与原对象在内存中完全独立。

特点：完全递归拷贝，彻底隔离。

副作用：比浅拷贝慢，且消耗更多内存（因为它创建了大量的重复对象）。


```
import copy
original = [[1, 2], 3]
deep = copy.deepcopy(original)
deep[0][0] = 'X'
print(f"原对象: {original}") # 输出: [[1, 2], 3] -> 不受影响
print(f"深拷贝: {deep}")    # 输出: [['X', 2], 3]
```

accumulate

```
from itertools import accumulate
nums = [1,2,3,4,5]
list(accumulate(nums)) # [1,3,6,10,15]
list(accumulate(nums,operator.mul)) # [1,2,6,24,120]
nums = [3,1,5,2,4]
list(accumulate(nums, max)) # [3,3,5,5,5]
```

常用集合操作对照表

术语	集合符号	位运算 (C++/Python)	集合示例	位运算示例 (1101 与 0111)
交集	$A \cap B$	<code>a & b</code>	$\{0, 2, 3\} \cap \{0, 1, 2\} = \{0, 2\}$	<code>1101 & 0111 = 0101</code>
并集	$A \cup B$	<code>`a</code>	<code>b`</code>	$\{0, 2, 3\} \cup \{0, 1, 2\} = \{0, 1, 2, 3\}$
差集	$A \setminus B$	<code>a & (~b)</code>	$\{0, 2, 3\} \setminus \{0, 1, 2\} = \{3\}$	<code>1101 & (~0111) = 1000</code>
对称差	$A \Delta B$	<code>a ^ b</code>	仅属于 A 或 B 的元素	<code>1101 ^ 0111 = 1010</code>
包含于	$A \subseteq B$	<code>(a & b) == a</code>	检查 A 是否为 B 的子集	<code>(0101 & 0111) == 0101 (True)</code>

格式说明符详解

- 对齐方式与宽度

说明符	含义	示例	输出
<	左对齐	f"{'hi':<6}"	'hi '
>	右对齐（默认）	f"{'hi':>6}"	' hi'
^	居中对齐	f"{'hi':^6}"	' hi '
数字	总宽度（字符数）	f"{123:6}"	' 123'
=	填充符后数字前	f"{42:+=5}"	'+ 42'

• 数字格式化

类型	含义	示例	输出
d	十进制整数	f"{123:d}"	123
f	浮点数（默认6位小数）	f"{3.14:f}"	3.140000
.nf	保留 n 位小数	f"{3.14:.2f}"	3.14
%	百分比（自动乘 100）	f"{0.85:%}"	85.000000%
.n%	百分比保留 n 位小数	f"{0.85:.1%}"	85.0%
e	科学计数法（小写 e）	f"{12345:e}"	1.234500e+04
E	科学计数法（大写 E）	f"{12345:E}"	1.234500E+04
g	自动切换普通/科学计数法	f"{12345:g}"	12345

• 千分位分隔符

```
print(f"{1234567:,}")          # 输出: 1,234,567
print(f"{1234567.89:,.2f}")   # 输出: 1,234,567.89
```

• 填充字符

默认填充格，你也可以自定义填充字符：

```
print(f"{'hi':*^10}")  # 输出: ***hi***
print(f"{42:0>5}")     # 输出: 00042
```

• 符号控制（整数/浮点）

符号	含义	示例	输出
+	总是显示正负号	f"{42:+}"	+42
-	仅负数显示负号	f"{42:-}"	42
空格	正数前留空，负数显示负号	f"{42: }"	42

• 字符串处理格式

```
text = "Python"
print(f"{text:.3}")      # 输出: Pyt (截取前3个字符)
print(f"{text:>10}")     # 宽度10, 右对齐
print(f"{text:*^10}")    # 居中, 用*填充: **Python**
```

• 多项组合（格式说明可以组合使用）

```
x = 3.14159
print(f"{x:>10.2f}")    # 宽度10, 右对齐, 保留两位小数
# 输出:          3.14
```

• 进制格式化

格式	含义	示例	输出
b	二进制	f"{10:b}"	1010
o	八进制	f"{10:o}"	12
x	十六进制（小写）	f"{255:x}"	ff
X	十六进制（大写）	f"{255:X}"	FF
#	显示前缀	f"{255:#x}"	0xff

查找dict的第k个key

可以用 itertools.islice 按需取（更省内存）

```
from itertools import islice
d = {"a": 1, "b": 2, "c": 3}
k = 1
key = next(islice(d.keys(), k, k+1))
print(key) # b
```

字符串大小写互换

用string.swapcase()来实现

想使用改变的set，但又不想真正改变原set

用集合差集运算

```
- 运算符（推荐）
a = {1, 2, 3, 4}
b = {2, 4}
result = a - b
print(result) # {1, 3}
print(a)      # {1, 2, 3, 4} (原 set 没变)
```

dict.setdefault(key, default)

这个是defaultdict用不了的选择

功能：

如果字典中存在 key，返回该 key 对应的 value。

如果字典中不存在 key，则将 key 添加到字典，并将其 value 设置为 default，然后返回 default。

```
d = {}
d.setdefault('x', []).append(10)
print(d) # {'x': [10]}
```

reversed(seq)

参数 seq：必须是序列类型 比如 list、tuple、str、range 等。

返回值：一个 反向迭代器（不是 list，而是 reversed 对象）。

```
nums = [1, 2, 3, 4]
rev = reversed(nums)
print(rev)                # <list_reverseiterator object>
print(list(rev))          # [4, 3, 2, 1]
```

find()用法(只能用于string)

```
s = "hello world"
print(s.find("world"))    # 6
print(s.find("o"))        # 4 (第一个 o 在索引 4)
print(s.find("python"))   # -1 (找不到)
print(s.find("o", 5))     # 7, 从索引 5 开始找, 找到第二个 o
print(s.find("o", 8))     # -1, 8 之后没有 o
print(s.find("o", 0, 5))  # 4, 只在 [0,5) 范围找
print(s.find("o", 5, 8))  # 7, 只在 [5,8) 范围找
```

s.replace(old, new, count)

old: 要被替换掉的子串

new: 新的子串

count (可选): 替换的最大次数, 省略时表示替换全部

```
s = "hello world world"
# 替换所有 "world" 为 "python"
print(s.replace("world", "python"))
# 输出: hello python python
# 只替换前 1 个
print(s.replace("world", "python", 1))
# 输出: hello python world
# 原字符串不变
print(s)
# 输出: hello world world
```

ord和chr

ord("A") = 65

ord("Z") = 90

ord("a") = 97

ord("z") = 122

chr(65) = "A"

函数形参

- 函数的形参引入的是不可变对象（如整数、字符串、元组），这些对象在函数内部修改时会创建新对象，必须通过参数传递才能跨调用保持更新。
可变对象 (列表、字典等) 可以直接修改，通常不需要作为参数传递：

```
def factorial(n: int) -> int:
    if n == 0:
        return 1
    return n * factorial(n - 1) # 必须传递n-1
```

- 当不同递归层级需要独立的状态副本时，必须通过参数传递：

```
def backtrack(i,path):
    mylist.append(path.copy())
```

常见 is 开头的方法及作用

方法名	功能描述	示例	结果
isupper()	判断字符串是否全部为大写字母	"ABC".isupper()	True
islower()	判断字符串是否全部为小写字母	"abc".islower()	True
isdigit()	判断字符串是否全部由数字组成	"123".isdigit()	True
isalpha()	判断字符串是否全部由字母组成	"abc".isalpha()	True
isalnum()	判断字符串是否由字母和数字组成	"abc123".isalnum()	True
isspace()	判断字符串是否全部由空白字符组成	" \t\n".isspace()	True
istitle()	判断字符串是否为标题格式 (首字母大写)	"Hello World".istitle()	True

" ".islower() 的结果是 False

```
print("123 !".upper())      # 输出: 123 !
print(" ".upper())          # 输出:   (还是空格)
```

upper() 只转换字母，不改变数字、空格和符号。

math

使用前要先导入：

```
import math
```

里面主要有：数学常量、基础函数、三角函数、指数对数函数、取整函数、特殊函数等。

```
math.pi      # 圆周率  $\pi = 3.14159...$   
math.e        # 自然常数  $e = 2.71828...$ 
```

```
math.pow(x, y)    # 幂运算  $x^y$   
math.factorial(n) # 阶乘  $n!$   
math.gcd(a, b)    # 最大公约数
```

```
math.ceil(x)      # 向上取整
```

```
math.exp(x)       #  $e^x$   
math.log(x)       #  $\ln(x)$ , 自然对数  
math.log10(x)     # 以 10 为底的对数  
math.log2(x)      # 以 2 为底的对数
```

```
math.sin(x)       # 正弦,  $x$  是弧度  
math.cos(x)       # 余弦  
math.tan(x)       # 正切  
math.asin(x)      # 反正弦 (返回弧度)  
math.acos(x)      # 反余弦  
math.atan(x)      # 反正切  
math.atan2(y, x)  # 用坐标  $(y, x)$  求角度
```

角度与弧度转换：

```
math.radians(deg) # 角度  $\rightarrow$  弧度  
math.degrees(rad) # 弧度  $\rightarrow$  角度
```

```
math.sinh(x)    # 双曲正弦
math.cosh(x)    # 双曲余弦
math.tanh(x)    # 双曲正切

math.comb(n, k)  # 组合数 C(n,k)
math.perm(n, k)  # 排列数 P(n,k)
```

from itertools import permutations

生成所有排列

```
text = "ABC"
result = list(permutations(text))
print(result)
# 输出: [('A', 'B', 'C'), ('A', 'C', 'B'), ('B', 'A', 'C'),
#        ('B', 'C', 'A'), ('C', 'A', 'B'), ('C', 'B', 'A')]
```

end

```
print("Hello", end="")
print("World")
# HelloWorld
```

正则（Regular Expression）

符号	含义	示例
.	匹配任意一个字符（除了换行）	a.c → "abc" , "axc"
^	匹配字符串开头	^abc → "abc123"
\$	匹配字符串结尾	abc\$ → "123abc"
[]	字符集，匹配其中任意一个	[abc] → "a" or "b" or "c"
[^]	否定字符集，匹配不在里面的字符	[^0-9] → 非数字
	或	abc xyz → "abc" 或 "xyz"
\	转义特殊字符	\. 匹配点 "."

符号	含义	示例
*	匹配 0 次或多次	a* → "", "a", "aaa"
+	匹配 1 次或多次	a+ → "a", "aaa"
?	匹配 0 次或 1 次	a? → "" 或 "a"
{n}	精确匹配 n 次	a{3} → "aaa"
{n,}	匹配 n 次或以上	a{2,} → "aa", "aaa", ...
{n,m}	匹配 n 到 m 次	a{2,4} → "aa", "aaa", "aaaa"

符号	含义	示例
\d	数字 [0-9]	\d{3} → "123"
\D	非数字	\D+
\w	单词字符 [a-zA-Z0-9_]	\w+
\W	非单词字符	\W+
\s	空白字符 (空格, 制表符, 换行)	\s+
\S	非空白字符	\S+

()：分组
(?:)：非捕获分组（不保存匹配结果）

例子：

```
import re
s = "ababab"
re.findall(r"ab+",s)      # ['ab','ab','ab']
re.findall(r"(?:ab)+",s)  # ['ababab']
re.findall(r"(ab)+",s)    # ['ab']
```

由于 (ab) 是一个捕获组，且后面跟着 +（匹配一次或多次），这个捕获组会不断被更新。它先捕获第一个 "ab"，然后是第二个 "ab"，最后是第三个 "ab"。最后一次捕获的内容会覆盖掉之前的内容。因此，最终返回的是最后一次捕获的 "ab"。

(\d{4})-(\d{2})-(\d{2})

匹配 "2025-10-04"

分组 1 = "2025", 分组 2 = "10", 分组 3 = "04"

```
import re
# 匹配
re.match(r"\d+", "123abc")    # 从开头匹配
re.search(r"\d+", "abc123")   # 搜索第一个匹配
re.findall(r"\d+", "a1b22c3") # ['1', '22', '3']
# 替换
re.sub(r"\d+", "X", "a1b22c3") # aXbXcX
# 分割
re.split(r"\s+", "a b c")     # ['a', 'b', 'c']
```

匹配形如 2025-10-04 的日期，并分别捕获年、月、日。

```
import re
text = "2025-10-04"
pattern = r"(\d{4})-(\d{2})-(\d{2})" # 数字要加{}
result = re.findall(pattern, text) # findall函数有两个形参
print(result) # [('2025', '10', '04')]
```

匹配常见邮箱地址，如：alice@example.com bob_123@pku.edu.cn

要求分组出 用户名 和 域名

```
import re
text = "alice@example.com bob.world-123@pku.edu.cn"
pattern = r"([\w.-]+)@([\w.-]+\.[a-zA-Z]{2,6})" # \w匹配字母数字和下划线 \.是专门匹配. (否则
result = re.findall(pattern, text)
print(result) # [('alice', 'example.com'), ('bob.world-123', 'pku.edu.cn')]
```

bin(x)

把整数 x 转换为二进制字符串。

格式固定为：

'0bxxxxx'

其中 0b 是前缀，表示“二进制”。